ML Project

Machin Learning

2026/4/17

Basem Fahad Al-Hazmi **441009273**

Waleed Hamoud Al-Harbi **441008303**

Abdullah Mohamed Al-Gurafi **441009503**

Dr. Ismail Domlo

Contents

Abstract

This project aims to develop an artificial intelligence model capable of predicting Customer Churn before it happens, which will provide companies with an opportunity to intervene proactively through this project, various things and steps have been tried from data exploration and cleaning through Feature Engineering to building pipelines for data processing and training models .The focus in this project was on Logistic Regression and improving its performance by adjusting Hyperparameter Tuning and adjusting Decision Threshold to maximize the scale F1 Score instead of Accuracy to suit the nature of unbalanced data

Introduction

Identify the problem

In today's business world, retaining existing customers is much less expensive than acquiring new ones. the problem that I tried to fix in this project Churn is that when a customer decides to terminate his subscription, the challenge is that this process does not happen suddenly, but is preceded by behavioral indicators .The goal was to train a model to capture these indicators.

Project objectives

i. analyzing the customer data set to extract behavioral and financial patterns
ii. building a software structure Pipeline for cleaning and processing data
iii. comparing different machine learning algorithms and choosing the best one
iv. Submit a final form with a minimum of False Negatives.

EDA

To ensure the accuracy of the results, a strict scientific methodology was followed and divided into the following stages

data exploration and cleaning (EDA & Data Cleaning)

I started by downloading the data and examining its structure, and then I realized that the quality of the model depends on the quality of the data, so I focused on several points:

1. *Missing Values analysis: I analyzed all the features and found many features that have missing values and these features are very important*
2. *Dealing with Outliers: I used Boxplots to visually find out the anomalous values in the features and then applied IQR mathematics to accurately determine the size of these values .Which helped me decide on a strategy to tackle it later.*
3. *Delete features :I deleted several features like*

(a) customer_id: because it never benefits the model

(b) weekly_minutes, usage_score_proxy: these features have been deleted because of their strong correlation, which brings them to the output of the same result and influence on the model

(c) support_tickets_90d: this feature has been deleted

Because it treats a person who uploaded 5 tickets in a year and a person who uploaded 5 tickets in a month with the same thing and makes them equal, so this problem was solved by Feature Engineering and then deleted

Methodology

Missing value

I analyzed all the features and found many features that contain missing values, and the total of these missing values was 24.23%, and this is about a quarter of the important data, which are missing values, and the feature that had the greatest impact is angagement_score at 9.30%, so it should be treated with caution .If you delete all rows with missing values, this will lead to the loss of other important values, so the best strategy should be chosen to compensate for the missing values without affecting the model .

	Missing Count	Missing Percent
engagement_score	279	9.30
avg_session_minutes	123	4.10
estimated_income	103	3.43
monthly_spend	92	3.07
last_login_days_ago	70	2.33
support_tickets_90d	60	2.00

Is Missing Value Random

In the following table. He compares 6 features with churn, where he showed that all the missing values are random, so I used the p-value column for comparison and showed all the features that they have p-vlue more than 0.05, which means that the missing values are random, for example monthly_spend has p-vlue 0.536, as well as you have the avg_session_minutes feature that has p-value P 0.562, which is considered to be greater than 0.05 is a big difference, so it has random missing values and the other features have the same principle.

Outliers

The outlier analysis showed that some numerical features contained noticeable extreme values. The highest outlier percentages appeared in **support tickets (last 90 days)**, **spend per session**, and **days since last login**, followed by **payment delays**, **tenure in months**, and **usage variability**. However, these values were not removed automatically because they may reflect real customer behavior rather than clear data errors. In churn prediction, unusual behavior can still be meaningful, especially for customers with irregular usage patterns or support activity. Therefore, we decided to inspect the outliers during the EDA stage and keep them unless there was strong evidence that they were invalid. This approach helped us preserve potentially useful information while avoiding unnecessary data loss.

	Feature	Outlier Count	Outlier %
8	support_tickets_90d	225	7.65
14	spend_per_session	215	7.17
10	last_login_days_ago	159	5.43
9	payment_delays_6m	101	3.37
2	tenure_months	100	3.33
12	usage_variability	83	2.77
5	avg_session_minutes	58	2.02
6	weekly_minutes	46	1.53
13	engagement_score	43	1.58
3	monthly_spend	41	1.41
7	usage_score_proxy	36	1.20
11	estimated_income	15	0.52
0	age	11	0.37
4	sessions_per_week	6	0.20
1	satisfaction_score	0	0.00
15	auto_renew	0	0.00

Chi-Square

We applied the **Chi-square test** to the categorical features to check their relationship with **churn**. The results showed that **contract type** had the strongest and most significant relationship with churn, while **region** was also statistically significant. The remaining categorical features had p-values above 0.05, which means they did not show strong evidence of a significant relationship with churn in this dataset.

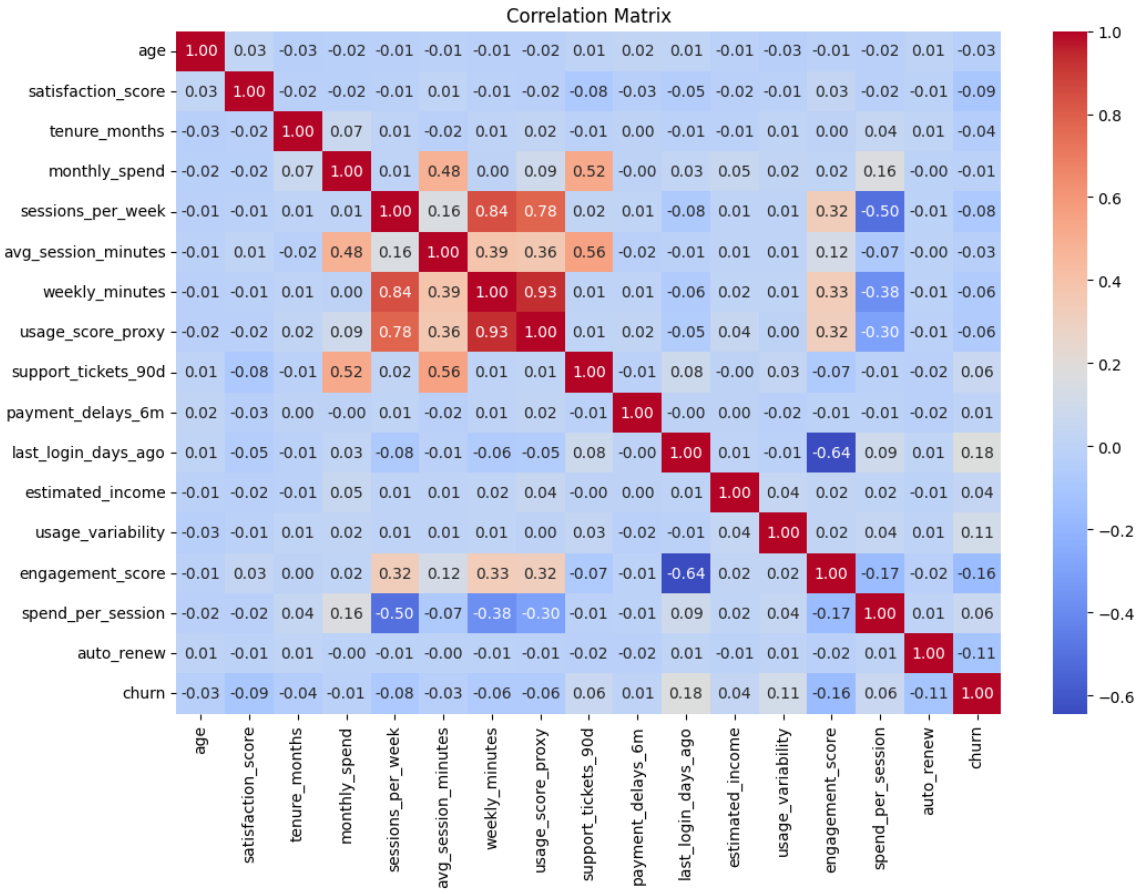
	Feature	Chi-Square	P-value
3	contract_type	170.2139	0.000000
0	region	15.5281	0.003722
5	promo_opt_in	2.1482	0.142735
4	payment_method	1.8319	0.400140
1	subscription_type	0.9839	0.611419
2	device_type	0.2986	0.861318

Correlation Matrix and Covariance Matrix

Correlation Matrix

We compute the correlation matrix to better understand the linear relationship between the numerical in the dataset. The results showed that some features were strongly related and were describing nearly the same aspect of customer behavior. **Weekly minutes** and **usage score proxy** had a very strong positive correlation of about **0.933**. Also, **sessions per week** had a strong positive correlation with **weekly minutes** at about **0.840** and with **usage score proxy** at about **0.780**. This indicates that these variables are highly redundant and mostly reflect the same usage pattern.

Another important relationship appeared between **last login days ago** and **engagement score**, which had a strong negative correlation of about **-0.643**. This means that customers who stay inactive for longer periods usually have lower engagement. This was a meaningful behavioral signal because inactivity and weak engagement are both logically connected to churn risk.



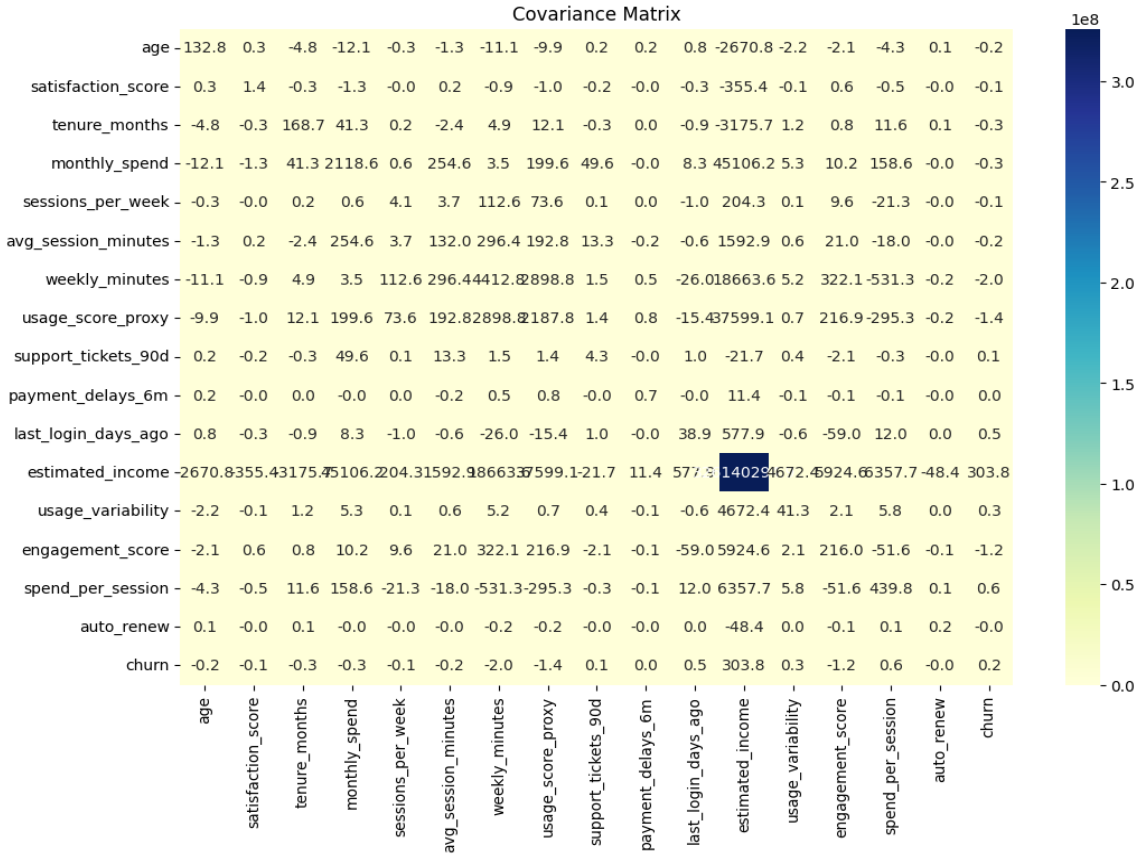
Overall, the correlation matrix helped us identify which features were overlapping, which features had the clearest relationship with churn, and which variables could be combined or removed during feature engineering. This made the later preprocessing and model selection steps more justified and more consistent with the structure of the data.

Covariance Matrix

We also computed the covariance matrix to examine how the numerical variables move together. The covariance results supported the same general trends observed in the correlation matrix, but they also showed an important difference: covariance is highly affected by the scale of the variables. For example, **weekly minutes** and **usage score proxy** had a large positive covariance of about **2898.81**, which confirms that these two variables increase together. In contrast, **last login days ago** and **engagement score** had a negative covariance of about **-58.97**, which means

that as inactivity increases, engagement tends to decrease. These patterns are fully consistent with the behavioral interpretation of the dataset.

At the same time, some covariance values were very large simply because the variables themselves had large numeric ranges. A clear example is **monthly spend** and **estimated income**, which had a covariance of about **45106.19**, even though their correlation was actually weak. This shows that covariance alone does not measure the strength of the relationship in a fair way across different variables, because it is not standardized like correlation.



For this reason, covariance was useful mainly for understanding the direction of movement between features, while correlation was more useful for comparing the strength of relationships and making feature selection decisions. In our project, the covariance matrix supported the same conclusions from the correlation matrix, but

the correlation matrix provided clearer evidence for removing redundant features and interpreting churn-related patterns.

Feature Engineering

Features Removed

Based on the **correlation matrix** and our feature comparison, we found that **weekly minutes**, **usage score proxy**, and **sessions per week** were describing nearly the same aspect of customer usage behavior. To reduce redundancy and avoid repeated information, we removed **weekly minutes** and **usage score proxy**, while keeping **sessions per week** as the representative usage feature.

We also removed **customer ID** because it is only an identifier and does not provide meaningful information for predicting churn. Keeping it would not improve the model and could add unnecessary noise.

In addition, we removed **support tickets (last 90 days)** after creating **support pressure**, since the engineered feature gave a more useful and interpretable representation of customer support activity relative to usage.

Features Added

Our final approach followed an **Original + Engineered** strategy, where we kept the most informative original variables and added a small set of engineered features supported by the **EDA** and **statistical testing**.

The feature **renewal timing indicator** was created from **contract type** and **tenure in months** to represent whether a customer is near a subscription renewal point. Since renewal periods are often decision points for continuing or canceling a service, this feature showed a meaningful relationship with churn and was a strong addition.

The feature **inactive engagement gap** was created from **days since last login** and **engagement score**. It was designed to capture customers who are both inactive and

weakly engaged, which gave a clearer signal of churn risk than using either variable alone.

The feature **stability gap** was created from **usage variability** and **engagement score**. This feature aimed to capture customers with unstable usage patterns and low engagement, which may reflect a weaker relationship with the service and a higher likelihood of churn.

Feature Test

	Feature	Groups	Churn Gap	Lift
7	cycle_months	{'1': 0.4503, '12': 0.1587, '3': 0.3343}	0.2916	2.84
1	inactive_engagement_gap	{'Low': 0.2728, 'Medium': 0.3337, 'High': 0.4707}	0.1979	1.73
0	is_renewal_time	{'0': 0.2365, '1': 0.4286}	0.1921	1.81
2	stability_gap	{'Low': 0.2944, 'Medium': 0.3219, 'High': 0.4553}	0.1609	1.55
3	support_pressure	{'Low': 0.3048, 'Medium': 0.3418, 'High': 0.4351}	0.1303	1.43
6	support_tickets_90d	{'Low': 0.3244, 'Medium': 0.3835, 'High': 0.4377}	0.1133	1.35
5	usage_score_proxy	{'Low': 0.4086, 'Medium': 0.3433, 'High': 0.334}	0.0746	1.22
4	weekly_minutes	{'Low': 0.403, 'Medium': 0.336, 'High': 0.347}	0.0670	1.20

We used a simple testing approach to measure how strongly each feature affects churn by dividing the feature values into groups and then comparing the churn rate across those groups. This helped us see whether the churn behavior changed clearly from one group to another.

We noticed that the engineered features were in fact useful. In particular, **cycle months**, **inactive engagement gap**, **renewal timing**, and **stability gap** showed clear differences in churn rates between groups. For example, customers in the higher-risk groups of **inactive engagement gap** and **stability gap** had noticeably higher churn rates than customers in the lower-risk groups. The feature **renewal timing** also showed a strong effect, since customers who were at a renewal point had a much higher churn rate than those who were not. In addition, **support pressure** showed a meaningful increase in churn across its groups, which suggests that customer problems relative to usage are also related to churn.

These results gave us more confidence that the engineered features were not random. Instead, they captured patterns that were actually connected to customer behavior

and churn risk. This supported our decision to keep these features in the final version of the dataset.

Data Preparation

We handled data cleaning through the preprocessing pipeline instead of cleaning the full dataset before splitting. Missing values in numerical features were imputed using the median, while missing values in categorical features were filled using the most frequent category. Numerical features were then scaled, and categorical features were encoded using one-hot encoding. These preprocessing steps were fitted only on the training data and then applied to the validation and test data. This approach was important to prevent data leakage, because using information from the test set during preprocessing could make the model performance look better than it really is. By learning all preprocessing parameters from the training set only, we made the evaluation more reliable and realistic.

We used the **median** for missing values in numerical features because it is more robust to outliers than the mean. In our dataset, some numerical variables showed skewness and extreme values during the EDA, so the median was a safer choice because it is less affected by unusually high or low values. This made the imputation more stable and more suitable for the actual distribution of the data.

Missing values in categorical features were filled using the most frequent category because categorical variables represent labels, not numeric values, and this method keeps the data consistent in a simple way. It is also suitable here because the amount of missing data was limited, so this approach allowed us to keep the rows while maintaining consistency in the categorical variables. Categorical features were encoded using one-hot encoding because the model cannot work directly with text categories. This method converts each category into binary columns without creating a false order between categories.

Model Comparison & Selection

We selected these models because they represent different approaches to classification and allowed us to compare both performance and behavior on our churn dataset. **Logistic Regression** was included because it is a standard and interpretable model for binary classification problems such as churn prediction. **Random Forest** was chosen because it can capture non-linear patterns and interactions between features through multiple decision trees. **Gradient Boosting** and **HistGradientBoosting** were included because boosting methods are often strong on tabular datasets and can improve performance by learning from previous errors step by step. **Extra Trees** was also tested because it is another tree-based ensemble model that adds more randomness, which sometimes helps improve generalization. Overall, we used these models to compare a simple interpretable method against stronger ensemble methods and then choose the most suitable one for our project.

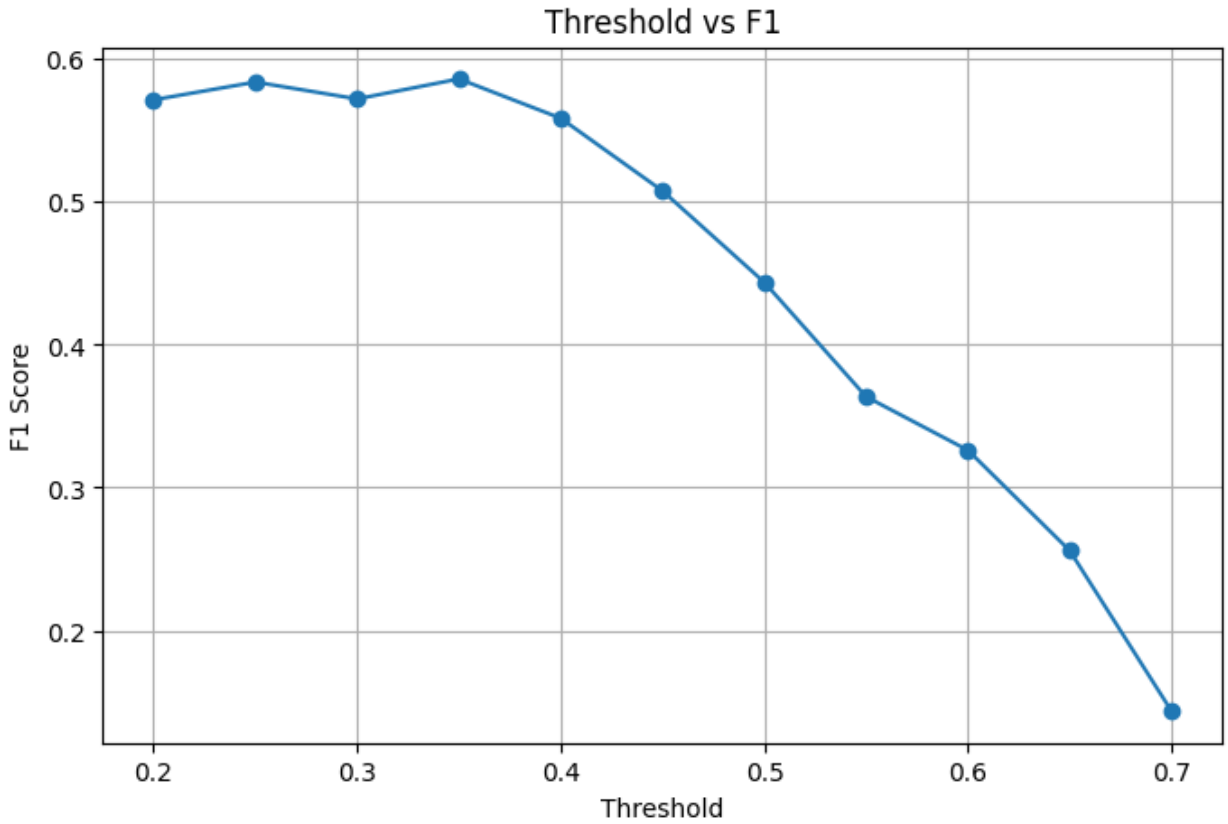
We chose **Logistic Regression** as our final model because it gave competitive results while being much easier to understand and explain than the more complex models. Even though some ensemble models achieved similar or slightly better performance, Logistic Regression provided a good balance between accuracy and interpretability. This was important for our project because the instructions emphasized reasoning, justification, and understanding the model, not only achieving the highest score.

	Model	Accuracy	Precision	Recall	F1
0	HistGradientBoosting	0.671667	0.568493	0.382488	0.457300
1	Logistic Regression	0.670000	0.568345	0.364055	0.443820
2	Gradient Boosting	0.668333	0.565217	0.359447	0.439437
3	Extra Trees	0.650000	0.527559	0.308756	0.389535
4	Random Forest	0.653333	0.537190	0.299539	0.384615

Model Evaluation

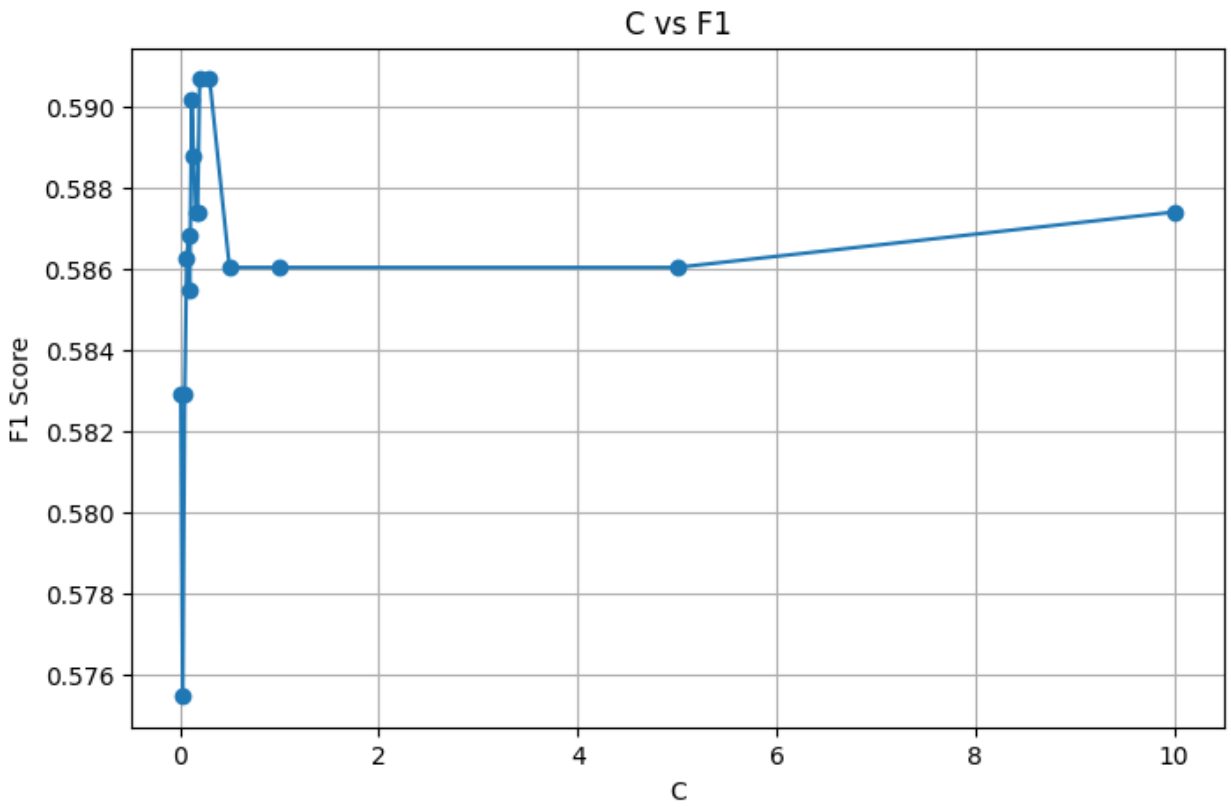
1. Best Threshold Selection

In this step, we focused on finding the most suitable **threshold** for the **Logistic Regression** model. We first tested the model using a relatively wide range of threshold values in order to observe how the performance changed, especially in terms of **F1-score**. After identifying the region that produced the strongest results, we gradually narrowed the range and tested smaller intervals around the best-performing values. This step-by-step process allowed us to choose the threshold that gave the most balanced performance for our churn classification task.



2. Best C Selection

After selecting a strong threshold value, we moved to tuning the **C** parameter of **Logistic Regression**, which controls the strength of regularization. We tested several values of **C** while keeping the selected threshold fixed, so that we could clearly measure the effect of this parameter on model performance. Similar to the threshold search, we started with a wider range of values and then reduced the search space gradually until we identified the most suitable value. This helped us find the level of regularization that produced the best result for the model.



Parameter Sensitivity Analysis

Our tuning results showed that the **threshold** had the strongest effect on model performance. The **F1-score** changed clearly across different threshold values, and the best results appeared in the range around **0.33 to 0.38**. When the threshold became higher, the F1-score dropped because the model became too conservative and missed more churn cases.

The **C** parameter also affected the model, but its impact was smaller than the threshold. Performance improved as **C** moved into the range around **0.15 to 0.25**, but after that the changes became very small. This means that C was important, but not as influential as the threshold.

We did not observe strong evidence of clear **overfitting** in the tested range. Instead, the results suggested **diminishing returns**, since testing more nearby values after the best region only produced very small improvements.

Overall, the experiments showed that **threshold mattered most**, **C had a moderate effect**, and after reaching the best region, further tuning gave only limited benefit. Based on this, we selected **C = 0.24** and **Threshold = 0.38** because they provided the best balance between **F1-score** and **accuracy**.

3. Combined Tuning of Threshold and C

In the final step, we tested **Threshold** and **C** together in a focused way. We took the best threshold value from the previous step and created a smaller range of values around it. We then did the same for the best **C** value by testing nearby values around the strongest result. This allowed us to evaluate the interaction between both parameters and check whether combining refined values could improve the model further. Through this process, we aimed to reach the best overall setting for the final version of our **Logistic Regression** model.

	C	Threshold	Accuracy	Precision	Recall	F1
30	0.25	0.33	0.616667	0.481481	0.747126	0.585586
31	0.15	0.34	0.625000	0.488462	0.729885	0.585253
32	0.16	0.34	0.625000	0.488462	0.729885	0.585253
33	0.15	0.36	0.639583	0.502075	0.695402	0.583133
34	0.17	0.36	0.637500	0.500000	0.695402	0.581731
35	0.24	0.38	0.654167	0.518018	0.660920	0.580808
36	0.25	0.38	0.654167	0.518018	0.660920	0.580808
37	0.16	0.31	0.600000	0.468310	0.764368	0.580786
38	0.16	0.36	0.637500	0.500000	0.689655	0.579710
39	0.22	0.36	0.637500	0.500000	0.689655	0.579710
40	0.15	0.32	0.604167	0.471223	0.752874	0.579646

We selected these values of **C** and **Threshold** because they provided the best balance between **F1-score** and **accuracy**. Although some other combinations achieved slightly higher values in one metric, this combination gave a more balanced overall performance across both measures. Since our goal was not only to maximize one metric alone, but to achieve a reliable and well-balanced model, we considered this setting the most suitable final choice.